

填空题

1.

拆分问题成多个子问题，递归解决子问题，将子问题合并成原问题的解

2.

减一法、减常因子法，减不定长因子

3.

实例化简、改变表现、问题化简

4.

减常因子法

5.

改变表现

6.

当 n 为偶数时：

$$a_n = (a^{n/2})^2$$

当 n 为奇数时：

$$a_n = a * (a^{(n-1)/2})^2$$

7. C. 分治法需合并子解，减治法无需合并

解答题

1. (1) 算法思想：

先将所有点从小到大排序，然后采用分治法将点集划分为左右两部分，分别递归求出左半部分和右半部分的最近点对距离。由于是一维有序点集，跨越边界的最近点对只需检查左半最后一个点和右半第一个点。最终取左半、右半和跨边界三者中的最小值作为全局最近距离。

(2) 解答过程：

原点集为 $P=[15, 3, 9, 22, 7, 1, 18, 4, 12, 26]$ 排序后得： $[1, 3, 4, 7, 9, 12, 15, 18, 22, 26]$

分成左右两部分：

左半：

$[1, 3, 4, 7, 9]$

右半：

$[12, 15, 18, 22, 26]$

左半继续分治，得到左半最近距离为：

$$dL=1$$

最近点对为 (3, 4)

右半继续分治，得到右半最近距离为：

$$dR=3$$

跨越左右总边界的距离为：

$$dM=12-9=3$$

因此全局最近距离为：

$$d=\min(1, 3, 3)=1$$

故最近点对为 (3, 4)，最近距离为 1。

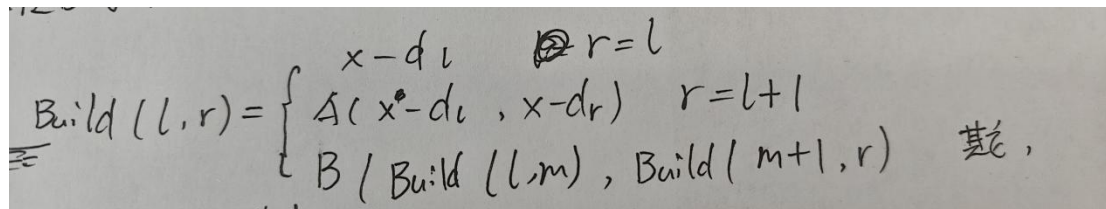
2.

$$P(x) = \prod_{i=1}^n (x - d_i)$$

因此问题转化为将 n 个一次多项式 $(x - d_i)$ 相乘。若按顺序逐个乘入，则总时间为 $O(n^2)$ 。为提高效率，采用分治法：

1. 将根集合分成左右两半；
2. 递归构造左半部分的多项式 $PL(x)$ ；
3. 递归构造右半部分的多项式 $PR(x)$ ；
4. 利用算法 B 将 $PL(x)$ 与 $PR(x)$ 相乘，得到总多项式。

递归定义如下：



$$\text{Build}(l, r) = \begin{cases} x - d_l & r = l \\ A(x - d_l, x - d_r) & r = l + 1 \\ B(\text{Build}(l, m), \text{Build}(m+1, r)) & \text{其他} \end{cases}$$

其中 $m = (l+r)/2$ (向下取整) 该算法的递推式为： $T(n) = 2T(n/2) + O(n \log n)$ 时间复杂度为： $O(n \log^2 n)$

填空题

1. 拆分问题成多个子问题, 递归解决子问题, 将子问题的解合并为原问题的解
2. 减一法, 减率因子法, 减不全长因子
3. 实例化简, 改变表现, 问题化简
4. 减率因子法.
5. 改变表现
6. $(a^{\frac{n}{2}})^2$ $(a^{\frac{n-1}{2}})^2 \cdot a$
7. C

解答题

1. (1) 先将所有点从小到大排序, 然后采用分治法将点集分成左右两部分, 分别递归求出左半部分与右半部分的最近点对距离, 再检查跨越边界的点对, 取这三者最小值作为全局最近距离

(2) ~~排序~~ [1, 3, 4, 7, 9, 12, 15, 18, 22, 26]

左半 [3, 3, 4, 7, 9] 右半 [12, 15, 18, 22, 26]

左半继续分治得到最近距离 $d_L = 1$

右半继续分治得到最近距离 $d_R = 3$

跨越左右边界距离为 $d_m = 3$

$$d = \min(1, 3, 3) = 1$$

故最近点对为 (3, 4) $d = 1$

$$2. P(x) = \prod_{i=1}^n (x - d_i)$$

1. 将根集合分成左右两半

2. 递归构造左半部分多项式 $P_L(x)$, 右半部分的多项式 $P_R(x)$

3. 利用算法B将 $P_L(x)$ 与 $P_R(x)$ 相乘

递归定义:

$$Build(l, r) = \begin{cases} x - d_l & r = l \\ \Delta(x - d_l, x - d_r) & r = l + 1 \\ B(Build(l, m), Build(m+1, r)) & \text{其他} \end{cases}$$

~~递推式~~ ~~$T(n) = 2T(\frac{n}{2}) + O(n)$~~ 其中 $m = \lfloor (l+r)/2 \rfloor$

递推式 $T(n) = 2T(\frac{n}{2}) + O(n \log n) \Rightarrow$ 时间复杂度 $O(n \log^2 n)$